M03 PROTECTED PRACTICE REVIEW

Open only after a genuine saved SQL attempt

Compare -> repair -> close -> changed query -> append evidence.

SF0 review - Set up MySQL Workbench and run your first query

1. Strong attempt should show

Connection test returns 1; course setup returns 12 customers and 48 orders; the transfer query returns exactly 7 rows with the requested columns; no UPDATE/DELETE/DROP/ALTER is used outside the controlled setup script.

2. Expected query/reasoning pattern

Run SELECT 1 AS connection_test; then USE da_sql_foundations; SELECT order_id, region, sales_amount FROM orders LIMIT 3; Confirm that the result grid shows three rows and that the table itself was not modified.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

Unknown database -> rerun the full setup script. Connection error -> verify the MySQL service and credentials before changing SQL. Wrong result -> make sure the correct schema is active and only the intended statement is selected/run.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF1 review - Relational basics, keys and NULL

1. Strong attempt should show

The NULL query contains only NULL scores. The NOT NULL query contains no NULL scores. COUNT(*) is greater than COUNT(satisfaction_score) on the course data. Your explanation says repeated customer_id values in orders can be legitimate because order grain is one row per order.

2. Expected query/reasoning pattern

SELECT order_id, customer_id, satisfaction_score FROM orders WHERE satisfaction_score IS NULL LIMIT 5; Notice that order_id should be unique, customer_id may repeat, and missing satisfaction is displayed as NULL rather than 0.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

If = NULL returns an unexpected/empty result, replace it with IS NULL. If you think repeated customer_id values are duplicates, restate the table grain before deleting or changing anything.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF2 review - SELECT, aliases and DISTINCT

1. Strong attempt should show

Alias labels appear only in the result. DISTINCT status + channel has no duplicate pairs. The result grain explanation matches the selected combination rather than the source-table grain.

2. Expected query/reasoning pattern

SELECT DISTINCT region, category FROM orders ORDER BY region, category; Say the result grain aloud before interpreting the rows.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

If DISTINCT appears to remove too much/little, check exactly which columns are in SELECT. If aliasing causes confusion, remember that the stored table schema is unchanged by AS.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF3 review - WHERE and filter logic

1. Strong attempt should show

Every returned transfer row satisfies status <> Cancelled, region in East/West and sales >= 25000. Inspect at least three rows and separately count qualifying rows.

2. Expected query/reasoning pattern

SELECT order_id, region, category, status, sales_amount FROM orders WHERE status='Completed' AND category='Technology' AND region IN ('North','South'); Inspect returned rows against all three conditions.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

Unexpected rows -> translate the WHERE clause back into plain language and check AND/OR parentheses. Missing rows -> check inclusive boundaries and quoted text. NULL-related issue -> use IS NULL/IS NOT NULL, not = NULL.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF4 review - ORDER BY and LIMIT

1. Strong attempt should show

The first 7 rows are the smallest eligible sales values under the specified tie rule; the no-LIMIT version confirms row 8 is not smaller than row 7 under the sort definition.

2. Expected query/reasoning pattern

SELECT order_id, sales_amount FROM orders ORDER BY sales_amount DESC LIMIT 5; Remove LIMIT temporarily to confirm the ordering continues downward.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

Wrong top-N -> inspect ORDER BY direction and whether LIMIT is applied after the correct filter. Ties appear random -> add an explicit secondary sort key.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF5 review - Aggregate functions

1. Strong attempt should show

Completed checkpoint remains 30 rows and 691500 sales. Transfer pending_count matches the independent count. Explain whether AVG(sales_amount) averages orders, customers or months - it averages the eligible rows.

2. Expected query/reasoning pattern

SELECT COUNT(*) AS completed_orders, SUM(sales_amount) AS completed_sales, AVG(delivery_days) AS avg_delivery_days FROM orders WHERE status='Completed'; The locked lab should give 30 completed orders and completed sales = 691500.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

Wrong aggregate -> restate the business question and eligible row set. Suspicious average -> check filter and source grain. COUNT mismatch -> check whether you used COUNT(*) or a nullable column.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF6 review - GROUP BY and result grain

1. Strong attempt should show

The transfer result grain is one row per channel among Completed orders. Sum of completed_count across channels equals 30 on the locked course data.

2. Expected query/reasoning pattern

SELECT category, COUNT(*) AS orders, AVG(delivery_days) AS avg_delivery_days FROM orders GROUP BY category ORDER BY avg_delivery_days DESC; State the result grain before interpreting 'slowest'.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

SQL error about nonaggregated columns -> check that every selected detail column is grouped or aggregated. Wrong grain -> remove/add grouping columns deliberately. Count mismatch -> check filter consistency and whether groups overlap.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF7 review - HAVING vs WHERE

1. Strong attempt should show

All source rows entering the transfer groups are Completed. Every surviving region meets the aggregate threshold. A version without HAVING shows the groups that were filtered out.

2. Expected query/reasoning pattern

SELECT channel, COUNT(*) AS completed_orders FROM orders WHERE status='Completed' GROUP BY channel HAVING COUNT(*) >= 8; Explain which condition acts before grouping and which acts after.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

Aggregate condition in WHERE -> move it to HAVING. Row-level condition in HAVING -> move it to WHERE unless there is a deliberate reason. If result is empty, keep the correct query and explain what the data means.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.

SF8 review - CASE for business classification

1. Strong attempt should show

Transfer band counts sum to 48. NULL rows land only in Unrated. Boundary scores 3 and 4 land in the intended bands. No row is silently excluded.

2. Expected query/reasoning pattern

Classify sales_amount >=30000 as Large, >=18000 as Medium, else Small. Separately classify delivery_days <=3 Fast, <=5 Standard, else Slow. Inspect exact boundary values 3, 5, 18000 and 30000.

3. What to compare

Compare the question, result grain and validation before comparing syntax. Equivalent correct SQL is allowed. Do not copy the teacher query merely to make your text look the same.

4. Likely repair

Wrong bands -> inspect WHEN order and boundary operators. NULL falls into ELSE -> test IS NULL explicitly before numeric comparisons. Business-rule uncertainty -> document/confirm thresholds rather than inventing them silently.

5. Fresh retry

Close this review. Solve the changed transfer query from a blank query tab. Run at least one independent validation. Save it as a new attempt in the evidence log.